

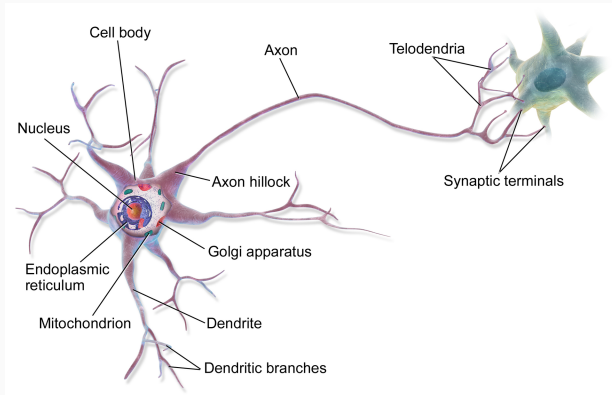
# DATA SCIENCE FOR FINANCE

## NEURAL NETWORKS

---

December 10

Neural Networks build on the idea of how neurons form networks in the brain



The Perceptron algorithm

$$f(x) = h(w \cdot x)$$

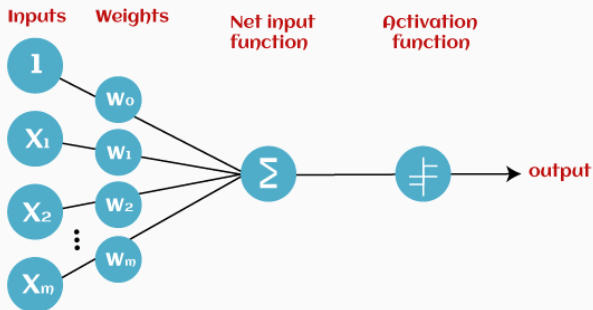
- $m$  is the number of features
- $x_i$  are the input variables.
- $w_i$  are the weights (parameters) that are being estimated/learned.

The Perceptron algorithm

$$f(\mathbf{x}) = h(\mathbf{w} \cdot \mathbf{x})$$

- $m$  is the number of features
- $x_i$  are the input variables.
- $w_i$  are the weights (parameters) that are being estimated/learned.
- $\mathbf{w} \cdot \mathbf{x} = \sum_{i=0}^m w_i x_i$
- $w_0$  is a bias, so  $x_0 = 1$
- $h(\cdot)$  might be some nonlinear function such as the sign function.

## The Perceptron algorithm



Generally

- Goal is to approximate a function  $\mathcal{F}(x) = \mathcal{Y}$

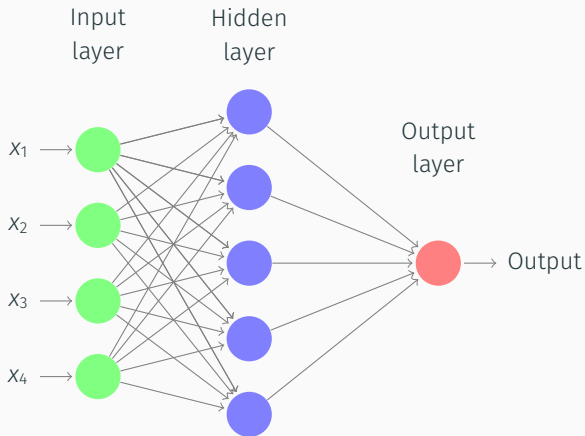
Generally

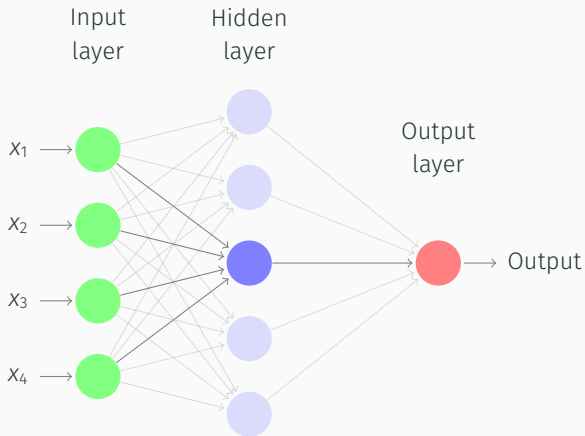
- Goal is to approximate a function  $\mathcal{F}(x) = \mathcal{Y}$
- The network defines a function that does so  $f(\mathbf{x}) = y$

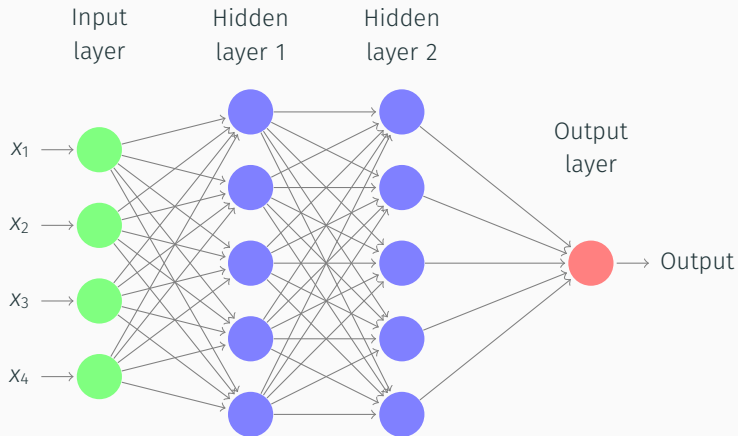
### Generally

- Goal is to approximate a function  $\mathcal{F}(x) = \mathcal{Y}$
- The network defines a function that does so  $f(\mathbf{x}) = y$
- This is done by combining several layers and neurons
- Many simple elements allows for high complexity









### Generally

- Goal is to approximate a function  $\mathcal{F}(x) = \mathcal{Y}$
- The network defines a function that does so  $f(\mathbf{x}) = y$
- This is done by combining several layers and neurons
- Many simple elements allows for high complexity

$$f(\mathbf{x}) = g \left( h^p \left( h^{p-1} \left( \dots \left( h^1(\mathbf{x}) \right) \right) \right) \right)$$

where  $g(\cdot)$  is the output layer

---

<sup>1</sup>dimensionality reduction, time series, images, speech recognition

### Generally

- Goal is to approximate a function  $\mathcal{F}(x) = \mathcal{Y}$
- The network defines a function that does so  $f(\mathbf{x}) = y$
- This is done by combining several layers and neurons
- Many simple elements allows for high complexity

$$f(\mathbf{x}) = g \left( h^p \left( h^{p-1} \left( \dots \left( h^1(\mathbf{x}) \right) \right) \right) \right)$$

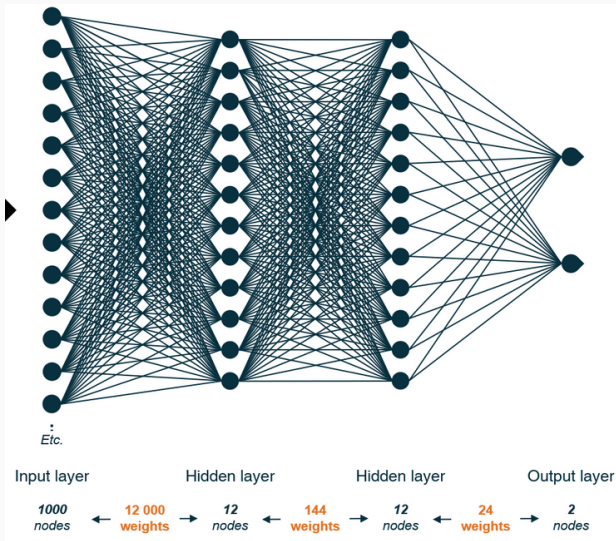
where  $g(\cdot)$  is the output layer

- Here  $h^i(\cdot)$  is the  $i$ 'th hidden layer in the network.
- Neural networks can be used for both supervised and unsupervised learning. We will look at some examples (for supervised learning) later.<sup>1</sup>

---

<sup>1</sup>dimensionality reduction, time series, images, speech recognition

# NEURAL NETWORKS - GENERALLY



## Learning & Activation Functions

- Learning is done using the method called backpropagation, see Deeplearning Chapter 6.5, (free online) for an explanation of it.
- Optimization done using the algorithm called Stochastic Gradient Descent, see Deeplearning Chapter 4.3 for more details.

### Learning & Activation Functions

- Learning is done using the method called backpropagation, see Deeplearning Chapter 6.5, (free online) for an explanation of it.
- Optimization done using the algorithm called Stochastic Gradient Descent, see Deeplearning Chapter 4.3 for more details.
- Usually ReLU is used,  $f(x) = \max(0, x)$
- See Deeplearning Chapter 6.1 for more details on activation functions.



### The Output Layer

- The activation function of the output layer depends on the application
- When used for example for regression, there is one output neuron
- For binary classification there is one neuron
- In the case of multi-class classification one would have  $K$  neurons, where  $K$  is the different classes

### The Output Layer

- The activation function of the output layer depends on the application
- When used for example for regression, there is one output neuron
- For binary classification there is one neuron
- In the case of multi-class classification one would have  $K$  neurons, where  $K$  is the different classes

### Mathematically

- Regression (Identity function):  $f(x) = x$
- Binary classification (Sigmoid):  $\sigma(x) = \frac{1}{1+e^{-x}}$ . Output is between 0 and 1. If output is greater than or equal to 0.5, then classify as 1, if below classify as 0.
- Multi-Class (Softmax):  $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$ . All output neurons are between 0 and 1, and they all sum to 1. The one with the highest value will be the classification.

### Loss Functions

- Generally:  $L = \sum_{i=1}^N L_i(\hat{y}_i, y_i)$
- Though  $L_i(\cdot)$  will depend on the application.

### Loss Functions

- Generally:  $L = \sum_{i=1}^N L_i (\hat{y}_i, y_i)$
- Though  $L_i (\cdot)$  will depend on the application.
- For regression, mean squared error (MSE),  $L_i (\hat{y}_i, y_i) = \frac{1}{n} (y_i - \hat{y}_i)^2$
- Or the mean absolute error (MAE),  $L_i (\hat{y}_i, y_i) = \frac{1}{n} |y_i - \hat{y}_i|$

## Loss Functions

- Generally:  $L = \sum_{i=1}^N L_i(\hat{y}_i, y_i)$
- Though  $L_i(\cdot)$  will depend on the application.
- For regression, mean squared error (MSE),  $L_i(\hat{y}_i, y_i) = \frac{1}{n} (y_i - \hat{y}_i)^2$
- Or the mean absolute error (MAE),  $L_i(\hat{y}_i, y_i) = \frac{1}{n} |y_i - \hat{y}_i|$
- Binary classification, Binary Cross-Entropy Loss:  
$$L_i = -(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$
- Multi-Class, Categorical Cross-Entropy Loss:  $L_i = -\sum_{j=1}^K y_{i,j} \log(\hat{y}_{i,j})$

There are several steps to go over before the data is (usually) ready to be used.

- Data types
- Data exploration
- Data cleaning
- Transformations & scaling

Things to keep in mind

- What type of data is it?
  - Timeseries, Cross sectional, panel data, images, text, etc.
- What is the goal?

### Things to keep in mind

- What type of data is it?
  - Timeseries, Cross sectional, panel data, images, text, etc.
- What is the goal?
- How does the goal effect the information we wanna extract from the variables?



### Things to keep in mind

- What type of data is it?
  - Timeseries, Cross sectional, panel data, images, text, etc.
- What is the goal?
- How does the goal effect the information we wanna extract from the variables?
- This should be kept in mind when performing cleaning, transformation, scaling etc.

Data exploration.

- Plotting: Line plots, histograms, scatter plots
- Basic statistics
- Correlation plots

### Data Cleaning

- Ensure variables have right type/format
- Consistency: Removing errors, ensure that stuff like this, K.U.L., KU Leuven, K.U.Leuven, KULEuven..., is not present

### Data Cleaning

- Ensure variables have right type/format
- Consistency: Removing errors, ensure that stuff like this, K.U.L., KU Leuven, K.U.Leuven, KULEuven..., is not present
- Removal observations that has missing values in one or more variables, removal outliers
- Or replace missing values, outliers with means, median, prior value. etc.

### Data Cleaning

- Ensure variables have right type/format
- Consistency: Removing errors, ensure that stuff like this, K.U.L., KU Leuven, K.U.Leuven, KULEuven..., is not present
- Removal observations that has missing values in one or more variables, removal outliers
- Or replace missing values, outliers with means, median, prior value. etc.

There is no exact way of how to do this. It depends on the situation, data, goal, and one has to figure-out what the best approach is.

Lets see it in practice

Often different types of transformations or scaling is needed

- Standardization:  $x_{new} = \frac{x - \mu}{\sigma}$  - Important: the mean and standard deviation from training data are saved and used to standardize the test data, or any other data that the model is used on.

Often different types of transformations or scaling is needed

- Standardization:  $x_{new} = \frac{x - \mu}{\sigma}$  - Important: the mean and standard deviation from training data are saved and used to standardize the test data, or any other data that the model is used on.
- Normalization:  $x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$  - As in Standardization, max and min from training set is used for the test set. - Good for prices, incomes etc.
- Log transformation - Potentially combine with one of the two above

Generally: Always apply standardization or normalization when working with neural networks

Transformation of non-numerical values: Dummyfication and other encodings

- Integer encoding:

*Low* = 1

*Medium* = 2

*High* = 3



Transformation of non-numerical values: Dummyfication and other encodings

- Integer encoding:

*Low* = 1

*Medium* = 2

*High* = 3

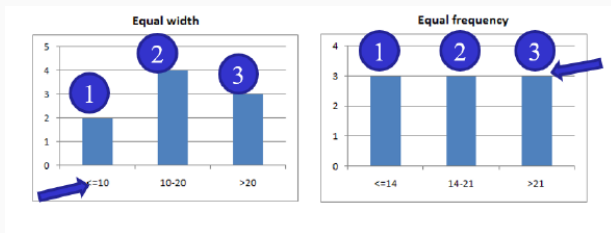
- One-Hot Encoding:

| Low | Medium | High |
|-----|--------|------|
| 1   | 0      | 0    |
| 0   | 1      | 0    |
| 0   | 0      | 1    |

**Table 1:** One-hot Encoding Representation for Categories Low, Medium, and High

Binning: Group range into categories

- Can be useful to treat outliers
- Equal width/interval binning versus equal frequency binning
- Beneficial if dealing with large sparse one-hot encoding vectors



Some data might already be in a numerical format, and may not need a standardization or normalization transformation, but might need another type of transformation for the model to extract the 'correct' type of information from it

Some data might already be in a numerical format, and may not need a standardization or normalization transformation, but might need another type of transformation for the model to extract the 'correct' type of information from it

- Data such as time, dates, months, etc.
- Dummy variables, could be relevant for example to indicate a season/quarter etc.
- Cyclical Features, such as hours, months, etc. transformed using sine/cosine transformation

Lets see it in practice.

See Scikit-Learn's User Guide chapter 6 for more on preprocessing

Approach:

- Split the data into training and test data.
- Usually 80 percentage is used for training and the remaining for testing
- Although it was mentioned under 'preprocessing', now the training data can be standardized.

### Approach:

- Split the data into training and test data.
- Usually 80 percentage is used for training and the remaining for testing
- Although it was mentioned under 'preprocessing', now the training data can be standardized.
- Test data is also scaled, using the same scaling values as the train data. This also applies in cross-validation!  $K - 1$  folds is the training set, the last fold is the validation/test set, and so it uses the same scaling values as the  $K - 1$  folds.
- Evaluation of performance is done using mean squared error, accuracy, F1, and so on, though depending on the application

For how to 'building', training and testing the model see the notebook, also see Scikit Learns guide and the API for regression.

There are many other things to tune to try to optimize the model's performance.

- Number of hidden layers and neurons
- Number of iterations: How many times should the model train on the training data? Too little might result in poor performance, too high might lead to overfitting and can be computationally heavy.



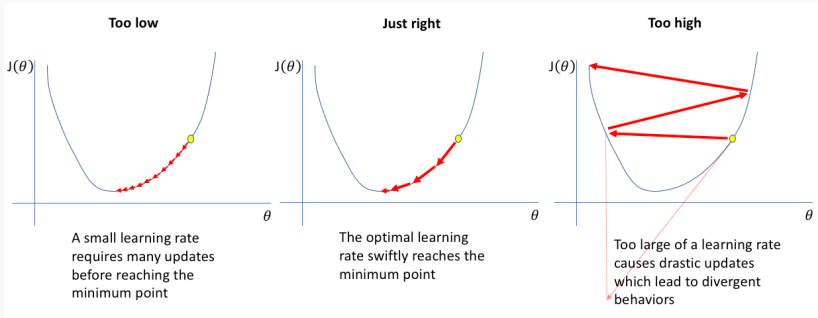
There are many other things to tune to try to optimize the model's performance.

- Number of hidden layers and neurons
- Number of iterations: How many times should the model train on the training data? Too little might result in poor performance, too high might lead to overfitting and can be computationally heavy.
- Batch size: How many observations should be used when updating the model? Too high can result in a overfitting, too low can bring a lot of noise. One should try to strick a balance.

There are many other things to tune to try to optimize the model's performance.

- L2 regularization term: Include a penalty to reduce overfitting and improves generalization
- Learning Rate: How big should the updates to the model be? Too small might lead to slow convergence or stuck in a local minimum, too high might lead to 'overshooting' the global minimum.

# NEURAL NETWORKS - TUNING PARAMETERS & OVERFITTING



### Pros:

- Can model complex and non-linear patterns that more simpler (linear) models are unable to
- Adaptability - Works well for a wide variety of data and applications
- Able to 'select' the relevant variable

### Cons:

- Can be computational expensive
- Hard to interpret
- Requires more data, than simpler models. Both for training and tuning
- Easy to overfit
- Often requires a lot of tuning